Several industries now run reinforcement learning in production, not just in simulation. I focus on systems where RL operates continuously in the field, affecting millions of real decisions, such as ride-hailing dispatch at DiDi, data center cooling at Google, hotel revenue management, and financial order execution. The chapter concludes with a simulation study on the bus engine replacement problem, demonstrating that RL matches and surpasses dynamic programming on a classical benchmark.

0.1 Ride-Hailing Dispatch

Each driver-passenger assignment changes the spatial distribution of available drivers, making ride-hailing dispatch a sequential optimization problem at a scale (tens of millions of daily rides at DiDi) where exact dynamic programming is intractable.

Qin et al. (2021) formalized DiDi's order dispatching as a semi-Markov decision process[1] where each driver is an independent agent. The state of a driver consists of location (discretized into hexagonal zones) and time (bucketed into intervals). The action is the order assigned to the driver, with the option to remain idle. The reward is the trip fare. State transitions are determined by trip destinations, as completing an order transports the driver from origin to destination, changing their spatial state. The stochasticity arises from future demand, which determines the available actions at each state. The per-driver value function $V^\pi(s)$ represents the expected cumulative fare a driver can earn from state s under dispatching policy π:

$$V^\pi(s) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^{\tau_k} r_k \mid s_0 = s, \pi\right], \tag{1}$$

where τ_k is the time to complete the k-th trip and r_k is the corresponding fare. The platform's objective is to maximize total driver income across the fleet, which decomposes into the sum of individual driver value functions.

Each dispatching window (a few seconds), the platform collects open orders and available drivers, constructs a bipartite graph, and solves a linear assignment problem. The edge weights are computed as the advantage of each driver-order match relative to the

[1] A semi-Markov decision process generalizes the standard MDP by allowing variable time between decisions. The discount factor γ^{τ_k} depends on the actual time elapsed τ_k rather than applying a fixed per-step discount.

driver's current state value.

$$w_{ij} = \hat{r}_i + \gamma^{\hat{\tau}_i} V(s'_j) - V(s_j), \tag{2}$$

where $\hat{r}_i$ is the predicted fare for order i, $\hat{\tau}_i$ is the estimated trip duration, and s'_j is the destination state. The value function is learned via temporal-difference methods from historical trip data. DiDi's Cerebellar Value Network (CVNet; Tang et al., 2019) uses hierarchical coarse-coding[2] with a multi-resolution hexagonal grid, enabling transfer learning across cities and robustness to data sparsity in low-traffic zones.

Production deployment across more than 20 Chinese cities demonstrated consistent improvements. Table 1 summarizes the reported gains from A/B tests using time-slice rotation, where algorithms alternate control of the platform in 3-hour blocks to avoid interference effects.

Table 1: DiDi dispatch deployment results from Qin et al. (2021) and Tang et al. (2019).

Metric	Improvement vs. Baseline	Scale
Total driver income	+0.5–2.0%	Tens of millions daily rides
Order response rate	+0.5–2.0%	Platform-wide
Fulfillment rate	+0.5–2.0%	Platform-wide

Li et al. (2019) addressed the coordination challenge using mean-field multi-agent RL. With thousands of drivers making simultaneous decisions, the full multi-agent state space is intractable. The mean-field approximation replaces individual driver states with an aggregate distribution, allowing each driver to condition on the density of nearby drivers rather than their exact locations. Experiments on DiDi data showed improved fleet utilization compared to single-agent baselines.

Han et al. (2022) reported complementary results from Lyft, where the dispatching system optimizes driver assignment and repositioning jointly. Their value decomposition architecture[3] assigns credit to individual driver decisions within the global objective. The production system demonstrated improvements in rider wait times and driver utilization, confirming that the RL dispatch paradigm generalizes across platforms and regulatory

[2]Coarse-coding represents a continuous state using overlapping receptive fields at multiple resolutions. At coarser levels, each tile covers a large region, capturing broad spatial patterns; finer levels capture local detail. The value function is the sum of features activated across all resolution levels.

[3]Value decomposition decomposes the platform's global matching objective into individual driver value functions, each estimable via temporal-difference learning. The global optimum is recovered by optimizing the sum of individual values.

environments.

At DiDi's volume, even 1% gains represent hundreds of thousands of additional completed rides per day, because dispatching is fundamentally a fleet positioning problem: today's assignments determine tomorrow's driver distribution.

0.2 Data Center Cooling

Data centers consume enormous amounts of energy, with cooling systems accounting for 30–40% of total power usage. Google applied machine learning to autonomous HVAC control, achieving substantial efficiency gains while maintaining temperature safety constraints.

Gao (2014) reported initial results from deploying neural network predictions to guide data center operations. The system used over 100 sensor readings, including temperatures at multiple locations, coolant flow rates, power draw, and external weather conditions, to predict Power Usage Effectiveness (PUE).[4] Operators used these predictions to adjust equipment setpoints. This assisted approach achieved a 40% reduction in cooling energy and a 15% improvement in overall PUE.

Lazic et al. (2018) subsequently deployed a fully autonomous system using model-predictive control with learned dynamics. The state space consists of temperatures, flow rates, and equipment states across multiple zones. Actions are setpoints for chillers, fans, and pumps. The system learns a neural network dynamics model $\hat{f}$ from operational data:

$$s_{t+1} = \hat{f}(s_t, a_t) + \epsilon_t. \tag{3}$$

At each control interval (5 minutes), the system optimizes over a horizon of future states using the learned model, subject to temperature constraints.

$$\min_{a_0,\ldots,a_H} \sum_{h=0}^{H} c_{\text{energy}}(a_h) \quad \text{s.t.} \quad T_{\min} \leq T_{\text{zone},i}(\hat{s}_h) \leq T_{\max} \; \forall i, h. \tag{4}$$

The MPC approach provides interpretability and safety guarantees that pure model-free RL cannot. Actions are projected onto the feasible set defined by temperature constraints, with uncertainty estimates from the learned model used to maintain conservative

[4] PUE is the ratio of total facility energy to IT equipment energy. A PUE of 1.0 means all energy goes to computing; the industry average is approximately 1.6.

margins. Human operators can override at any time. Table 2 summarizes the reported performance.

Table 2: Google data center cooling results from Gao (2014) and Lazic et al. (2018).

Metric	Improvement	Method
Cooling energy	-40%	Neural network predictions
PUE	-15%	Assisted operation
Additional efficiency	-12%	Autonomous MPC
Temperature violations	0	Safety constraints

The system operates continuously across multiple data centers, processing millions of sensor readings per day and controlling thousands of setpoints. The learned model adapts online to seasonal variations, equipment degradation, and workload changes.

0.3 Hotel Revenue Management

Hotels face a dynamic pricing problem with perishable inventory, since rooms unsold by check-in yield zero revenue. The state (t, I, d) consists of days until check-in, remaining rooms, and a demand intensity indicator. Actions are discrete price levels. Booking arrivals follow a Poisson process with intensity depending on demand conditions; each arrival books with probability decreasing in price. The Bellman equation is:

$$V_t(I, d) = \max_{p \in \mathcal{A}} \mathbb{E}\left[p \cdot B + V_{t-1}(I - B, d') \mid I, d, p\right], \tag{5}$$

where B is the number of bookings at price p and d' is the next-period demand state.

Gallego and van Ryzin (1994) established the analytical benchmark for dynamic pricing with Poisson demand. Under their intensity control framework, the optimal price trajectory can be characterized when demand follows specific functional forms. The Gallego-van Ryzin solution provides a theoretical upper bound, and any practical pricing system should approach this performance in settings where the model assumptions hold.

Chen et al. (2023) applied deep Q-learning to hotel revenue management, training on simulated booking data under parameterized demand models. The DQN agent uses state features (normalized time, normalized inventory, demand indicators) and outputs Q-values for each price level. Table 3 compares performance against baselines on held-out test periods.

Table 3: Hotel revenue management results from Chen et al. (2023).

Method	Mean Revenue	vs. Fixed Price
Fixed price	2,847	–
Myopic pricing	2,962	+4.0%
EMSR-b heuristic	3,156	+10.9%
Exact DP	3,241	+13.8%
DQN	3,198	+12.3%

The DQN agent achieves 98.7% of the DP optimal revenue while scaling to capacities where exact DP is infeasible.[5] Training time is nearly constant in hotel capacity, whereas DP computation grows linearly. The learned policies exhibit interpretable patterns, with prices increasing as inventory depletes and as check-in approaches, with larger adjustments during high-demand periods.

0.4 Financial Order Execution

The theoretical benchmark for optimal execution is the Almgren-Chriss framework (Almgren and Chriss, 2001), which derives optimal deterministic schedules under linear impact assumptions. A trader liquidating Q shares over T periods faces a tradeoff between timing risk and market impact. With risk-aversion parameter λ, price volatility σ, and temporary impact coefficient η, the optimal remaining inventory at time t follows the hyperbolic sine schedule:

$$x^*(t) = Q \cdot \frac{\sinh\big(\kappa(T-t)\big)}{\sinh(\kappa T)}, \quad \kappa = \sqrt{\frac{\lambda\sigma^2}{\eta}}. \tag{6}$$

This deterministic trajectory is the benchmark any adaptive method must beat. It prescribes front-loading or back-loading depending on the risk-impact balance, but cannot respond to realized order flow or spread dynamics.

Nevmyvaka et al. (2006) applied tabular Q-learning to execution on real limit order book data from NASDAQ stocks.[6] The state at time t is $s_t = (t, q_t, \psi_t, \Delta_t)$, where $t \in \{1, \ldots, T\}$ is time remaining, $q_t \in \{0, 1, \ldots, Q\}$ is inventory remaining, ψ_t is the discretized bid-ask spread, and Δ_t is the discretized signed volume imbalance.[7] Actions

[5] EMSR-b (Expected Marginal Seat Revenue, variant b) is the standard heuristic in airline and hotel revenue management. It computes booking limits by comparing the expected marginal revenue of reserving a unit for a higher fare class against selling it now.

[6] The dataset covers 500 trading days of millisecond-level limit order book snapshots for AMZN, QCOM, and NVDA. The state space contains approximately 10,000 states; the horizon is $T = 60$ seconds with discount $\gamma = 1$.

[7] Signed volume imbalance is the difference between buy and sell order volume near the best prices, normalized by total volume; positive imbalance typically predicts short-term price increases.

$a_t \in \{0, \delta, 2\delta, \ldots, q_t\}$ specify shares to execute in the current interval. The reward is the negative per-period slippage contribution:

$$r_t = -a_t\Big(p_t^{\text{exec}} - p_0\Big), \tag{7}$$

where p_0 is the mid-quote at order arrival and p_t^{exec} is the average execution price. Total implementation shortfall is $IS = -\sum_{t=1}^{T} r_t/Q$.[8] Because the objective is cost minimization, the Q-learning update uses min over next-period actions:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_t + \gamma \min_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\Big]. \tag{8}$$

The agent learns to condition on market microstructure signals: trading aggressively when spreads are narrow, waiting when order flow predicts favorable price movement, and accelerating near the deadline.

Table 4: Execution results from Nevmyvaka et al. (2006) on NASDAQ stocks. TWAP is the time-weighted average price baseline (equal-sized trades at uniform intervals).

Stock	RL vs. TWAP	RL vs. Almgren-Chriss	Data
AMZN	−18.2%	−14.6%	500 days LOB
QCOM	−22.1%	−19.3%	500 days LOB
NVDA	−15.8%	−12.1%	500 days LOB

The RL agent reduces execution costs by 12–19% over Almgren-Chriss, and several investment banks have subsequently deployed production execution algorithms built on these ideas.

0.5 Supply Chain Inventory Management

Multi-echelon inventory systems coordinate ordering decisions across supply chain stages, where each stage's order becomes the next stage's incoming shipment. A retailer orders from a warehouse, which orders from a distribution center, which orders from a factory. The sequential nature creates complex dependencies, since an order placed upstream today affects downstream availability many periods later. The state space grows exponentially in the number of echelons, with K stages each having M possible inventory

[8] Implementation shortfall measures the total cost of executing a trade relative to the benchmark mid-quote at arrival. It captures market impact, timing cost, and opportunity cost of unexecuted shares.

levels yielding M^K states. Classical inventory theory provides elegant solutions for special cases, most notably the echelon base-stock policy of Clark and Scarf (1960).

The state $s = (I_1, \ldots, I_K)$ records on-hand inventory at each stage, where stage 1 faces customer demand and stage K is the most upstream. The action $q \in \{0, 1, \ldots, Q_{\max}\}$ is the order quantity placed at stage K. Demand $D_t \sim F_D$ arrives at stage 1 each period; unfilled demand is backordered. Shipments flow downstream, with stage k receiving what stage $k+1$ shipped in the previous period. The per-period cost combines holding, backorder, and ordering components.

$$c_t = h \sum_{k=1}^{K} I_k^+ + b \cdot (D_t - I_1)^+ + c_o \cdot q_t, \tag{9}$$

where $I_k^+ = \max(0, I_k)$ is on-hand inventory, $(x)^+ = \max(0, x)$, h is holding cost per unit, b is backorder cost per unit, and c_o is ordering cost. The objective is to minimize expected discounted total cost.

Gijsbrechts et al. (2022) conducted a systematic evaluation of deep RL against classical base-stock policies across lost sales, dual sourcing, and multi-echelon configurations. The classical benchmark is the echelon base-stock policy, in which each stage k maintains an echelon inventory position and orders to bring this position to a target level S_k.[9] For single-echelon systems with backorders, the optimal base-stock level is given by the newsvendor critical fractile $S^* = F_D^{-1}(b/(b+h))$. Table 5 summarizes results from their multi-echelon experiments.

Table 5: Multi-echelon inventory results from Gijsbrechts et al. (2022).

Echelons	DRL Cost Gap vs. Base-Stock	States	Convergence
2	+1.2%	$\sim 10^2$	Achieved
3	+6.8%	$\sim 10^3$	Achieved
4	+12.3%	$\sim 10^5$	Partial
6	—	$\sim 10^8$	Failed

Where classical solutions exist, RL underperforms: the base-stock policy remains highly competitive when properly calibrated, and DRL required millions of training transitions to approach performance levels achievable through closed-form calculation. RL

[9] Formally, the echelon inventory position at stage k equals on-hand inventory at k plus all inventory at downstream stages $1, \ldots, k-1$ plus in-transit inventory, minus backorders. The echelon base-stock policy of Clark and Scarf (1960) is optimal for serial systems with linear costs and backorders.

struggles particularly with multi-echelon coupling, where upstream orders affect downstream costs many periods later.[10] The value proposition for RL lies in problems where analytical solutions are unavailable: non-stationary demand, complex operational constraints, or cost structures that do not admit tractable decomposition.

0.6 Real-Time Bidding

Real-time bidding for display advertising presents a budget pacing problem: an advertiser must allocate a fixed budget B_0 across a campaign of T auctions to maximize total conversions. Each impression is a second-price auction; the advertiser submits a bid and pays the second-highest bid if they win. The challenge is that bidding aggressively depletes budget early, while conservative bidding leaves value on the table.

Wu et al. (2018) formalized this as an MDP with state $s_t = (B_t, t, w_t)$, where B_t is remaining budget, t is auctions remaining, and w_t is the recent win rate. The action is a bid multiplier $\lambda_t \in [\lambda_{\min}, \lambda_{\max}]$, so the actual bid is $b_t = \lambda_t \cdot \bar{b}$, where $\bar{b}$ is a base bid calibrated to the estimated value-per-impression. The clearing price c_t is drawn from a distribution F_c estimated from historical data; the advertiser wins if $b_t \geq c_t$. The per-auction reward is:

$$r_t = \mathbb{1}\{b_t \geq c_t\} \cdot \mathrm{cvr}_t, \tag{10}$$

where $\mathrm{cvr}_t \in \{0, 1\}$ is the conversion indicator. Budget evolves as $B_{t+1} = B_t - \mathbb{1}\{b_t \geq c_t\} \cdot c_t$, and the episode terminates when $B_t \leq 0$ or $t = 0$.[11]

The agent is a deep Q-network trained on a simulator calibrated to production RTB data. Table 6 reports performance relative to rule-based pacing baselines. The DQN agent improves total conversions by conserving budget during high-competition periods and bidding aggressively when clearing prices are low, a pattern the rule-based approaches cannot learn.

[10] Credit assignment refers to the difficulty of determining which past actions caused a delayed reward or cost. In multi-echelon systems, an upstream ordering decision today may not affect customer-facing costs for many periods, making it difficult for RL to learn the causal connection.

[11] The state space is discretized into budget bins and time bins. Wu et al. (2018) augment the state with bid landscape features derived from historical clearing price distributions, giving the agent information about current market competitiveness.

Table 6: Real-time bidding results from Wu et al. (2018). Performance is relative to linear pacing on a simulator calibrated to production data.

Method	Conversions vs. Linear Pacing	Budget Utilization
Linear pacing	baseline	98%
Hard threshold	−8.3%	91%
Proportional (ECPC)	+4.1%	97%
DQN (Wu et al., 2018)	+11.7%	99%

0.7 Simulation Study: Bus Engine Replacement

I conclude with a simulation demonstrating that RL matches dynamic programming on a classical economics benchmark. The bus engine replacement problem, introduced by Rust (1987), models a fleet manager's monthly decision whether to replace engines based on accumulated mileage. Replacement incurs a fixed cost but resets mileage to zero; continued operation incurs maintenance costs increasing in mileage.

I extend the single-engine problem to a fleet of N engines with a capacity constraint limiting replacements per period. The state $s = (m_1, \ldots, m_N)$ records discretized mileage for each engine. Actions are subsets of engines to replace, subject to the capacity constraint. The per-period cost is $c(s, a) = \alpha \sum_i m_i + \beta|a|$, where α is the operating cost per unit mileage and β is the replacement cost.[12] Mileage evolves deterministically; replaced engines reset to $m = 0$; others increment by one bin.[13]

Table 7 compares dynamic programming, DQN, and heuristic baselines across fleet sizes.

Table 7: Bus engine replacement results across fleet sizes.

N	$\lvert\mathcal{S}\rvert$	DP Time	DP Return	DQN Return	Thresh(3)	Never Replace
1	6	0.06s	−55.1	-55.1 ± 0.1	−55.7	−94.0
2	36	0.06s	−110.3	-110.0 ± 0.3	−111.4	−187.7
3	216	0.59s	−165.3	-165.0 ± 0.1	−168.3	−282.1
4	1,296	6.93s	−220.0	-220.8 ± 0.1	−223.8	−376.0
5	7,776	81.79s	−275.7	-276.2 ± 0.1	−279.5	−469.7
6	46,656	—	—	-331.7 ± 1.1	−336.6	−562.8

[12] The mileage-dependent operating cost $\alpha \sum_i m_i$ follows Rust's original specification $c(x, \theta_1) = \theta_{11} x$, creating a non-trivial threshold replacement policy. The fleet extension uses deterministic mileage increments to isolate the combinatorial scaling challenge that arises from the joint state of multiple engines.

[13] With $M = 6$ mileage bins, the state space is 6^N: 1,296 states at $N = 4$, 7,776 at $N = 5$, 46,656 at $N = 6$. Value iteration is feasible for $N \leq 5$.

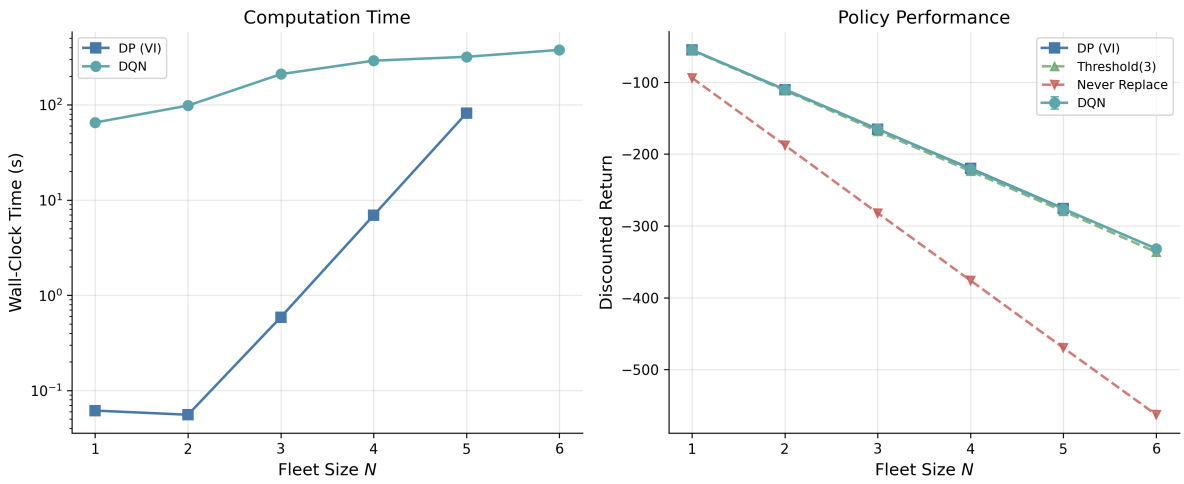

Figure 1: Bus engine replacement benchmark. Left: computation time vs. fleet size (log scale). Right: discounted return vs. fleet size for DP, DQN, and heuristic baselines.

For $N = 1$ through 5 where both methods are computable, DQN matches DP within 1% of the optimal discounted return. At $N = 6$ (46,656 states), DP is infeasible but DQN produces a policy. The threshold heuristic, which replaces engines above a mileage cutoff, provides a reasonable baseline but cannot account for capacity constraints or the joint state of multiple engines. The never-replace heuristic performs poorly due to accumulated mileage costs, confirming that the cost structure creates a non-trivial replacement decision.

References

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–40, 2001.

Yiwei Chen, Guillermo Gallego, and Tao Li. A reinforcement learning approach for hotel revenue management. *Journal of Operations Management*, 2023.

Andrew J. Clark and Herbert Scarf. Optimal policies for a multi-echelon inventory problem. *Management Science*, 6(4):475–490, 1960.

Guillermo Gallego and Garrett van Ryzin. Optimal dynamic pricing of inventories with stochastic demand over finite horizons. *Management Science*, 40(8):999–1020, 1994.

Jim Gao. Machine learning applications for data center optimization. Technical report, Google, 2014.

Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, and Dennis J. Zhang. Can deep reinforcement learning improve inventory management? Performance on dual sourcing, lost sales, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3):1349–1368, 2022.

Xiao Han, Weijian Zhang, Jiaxin Wang, Fan Zhang, and Jieping Ye. A better match for drivers and riders: Reinforcement learning at Lyft. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2927–2936, 2022.

Nevena Lazic, Tyler Lu, Craig Boutilier, Moonkyung Ryu, Eehern Wong, Binz Roy, and Greg Imwalle. Data center cooling using model-predictive control. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Minne Li, Zhiwei Qin, Yan Jiao, Yaodong Yang, Zheng Gong, Jun Wang, Changjie Wang, Gauge Wu, and Jieping Ye. Efficient ridesharing order dispatching with mean field multi-agent reinforcement learning. In *Proceedings of the 2019 World Wide Web Conference (WWW)*, pages 983–994, 2019.

Yuriy Nevmyvaka, Yi Feng, and Michael Kearns. Reinforcement learning for optimized

trade execution. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 673–680, 2006.

Zhiwei Qin, Xiaocheng Tang, Yan Jiao, Fan Zhang, Zhe Xu, Hongtu Zhu, and Jieping Ye. Ride-hailing order dispatching at DiDi via reinforcement learning. *INFORMS Journal on Applied Analytics*, 51(3):272–286, 2021.

John Rust. Optimal replacement of gmc bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033, 1987.

Xiaocheng Tang, Zhiwei Qin, Fan Zhang, Zhaodong Wang, Zhe Xu, Yintai Ma, Hongtu Zhu, and Jieping Ye. A deep value-network based approach for multi-driver order dispatching. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1780–1790, 2019.

Di Wu, Xiujun Chen, Xun Yang, Hao Wang, Qing Tan, Xiaoxun Zhang, Jian Xu, and Kun Gai. Budget constrained bidding by model-free reinforcement learning in display advertising. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1443–1451, 2018.